Trustless RPC.
A verifiable path to Ethereum.

Direct consensus and execution P2P. Proof-backed state. Local EVM execution.

01 · TRUST ANCHOR
Recent checkpoint
Finalized beacon root supplied explicitly or by a chosen HTTPS provider.

→

02 · BOOTSTRAP
Committee proof
Merkle proof binds the 512 committee keys to the checkpoint state.

→

03 · CONSENSUS
Signed beacon head
Verify aggregate BLS signatures, slots, participation and fork domain.

→

04 · COMMITMENT
Execution block
Verify its beacon commitment, then the execution roots and hashes.

→

05 · APPLICATION
Verified RPC
Check state / storage proofs and code hashes. Run calls locally.

342 / 512

Finalized head
Committee supermajority + finality proof.
Use for actions that must be irreversible.

LATEST

Authenticated optimistic head
Adaptive participation threshold.
Timely for wallets; can reorganize.

Trust-minimized: checkpoint + honest committee participation remain assumptions. Full block validity rests on Ethereum consensus. Timeout committee recovery has weaker assumptions.

Consensus specification Verifier implementation

P2P data. Local verification.

Fetch the authenticated head and the witnesses a request needs. No upstream execution JSON-RPC.

CONSENSUS NETWORK · LIBP2P

Beacon light-client data

Bootstrap	Checkpoint header, committee keys and committee proof.
Live gossip	Optimistic and finality updates, signatures and finality branches.
Recovery	Committee updates and next-committee proofs through request/response.

BLS + SSZ/Merkle verification · TCP/Noise/yamux or authenticated QUIC

EXECUTION NETWORK · RLPX / ETH / SNAP

Execution data on demand

ETH	Block headers, bodies, transactions and receipts.
SNAP	Account/storage ranges, Merkle witnesses and contract bytecode.
Local EVM	Calls and gas estimates execute against proven state; changes are discarded.

Header hashes + body/receipt roots + state proofs + code hashes

Find peers, then authenticate data

Both stacks use discv5. EL also uses signed DNS discovery at `snap.<network>.ethdisco.net`. Gossip follows heads; polling repairs gaps.

Availability remains a peer dependency

Peers can withhold data or stall a request. Invalid signatures, proofs or hashes are rejected against the authenticated commitments.

Wallet-ready RPC, executed locally.

23 ordinary methods over HTTP and WebSocket · 2 additional WebSocket methods

JSON-RPC 2.0 · port 8545

Identity & verified state

7 METHODS

`eth_chainId`
`eth_blockNumber`
`eth_getBalance`
`eth_getTransactionCount`
`eth_getStorageAt`
`eth_getCode`
`eth_getProof`

Simulation & fees

5 METHODS

`eth_call`
`eth_estimateGas`
`eth_feeHistory`
`eth_gasPrice`
`eth_maxPriorityFeePerGas`

Nethermind EVM simulation.
Fee suggestions use verified headers and receipts.

Blocks & counts

4 METHODS

`eth_getBlockByNumber`
`eth_getBlockByHash`
`eth_getBlockTransactionCountByNumber`
`eth_getBlockTransactionCountByHash`

Authenticated headers and verified block bodies.

Transactions & receipts

5 METHODS

`eth_getTransactionByBlockNumberAndIndex`
`eth_getTransactionByBlockHashAndIndex`
`eth_getTransactionByHash`
`eth_getTransactionReceipt`
`eth_getBlockReceipts`

Hash lookup uses a bounded recent/observed index.

Logs & signed relay

2 METHODS

`eth_getLogs`
`eth_sendRawTransaction`

Verify receipt-backed logs.
Validate already-signed transactions, then relay through execution P2P.

No signing keys. A relay hash is not proof of inclusion.

WebSocket subscriptions

2 METHODS

`eth_subscribe`
`eth_unsubscribe`

newHeads — authenticated execution tips
logs — verified events, with reorg removal notifications

`ws://127.0.0.1:8545/ws`

latest = optimistic, reorgable **finalized** = proven finalized state History and lookup are explicitly bounded.

Run guide & RPC contract RPC implementation

React to authenticated heads.

Use subscriptions today. Add a native consumer at the verified publication boundary.

External handler

AVAILABLE NOW

Connect to `ws://127.0.0.1:8545/ws`

```
{"jsonrpc":"2.0","id":1,
 "method":"eth_subscribe","params":["newHeads"]}
```

newHeads: compare hashes and ancestry to detect reorgs.
logs: orphaned events carry `removed: true`.



Undo effects from the old branch; apply the replacement branch.

In-process handler

PROPOSED EXTENSION

1. Verify consensus update and execution commitment.
2. Persist verified progress, then publish the head.
3. Dispatch to a bounded asynchronous custom worker.

> `oldHead → commonAncestor → newHead`
>
> Reuse the existing removed/added branch logic for explicit reorg callbacks. Make it independent of WebSocket subscriber interests.

The publication point exists. Public native callbacks and automatic plugin loading are not wired yet.

Durable consumer cursor | Gap recovery | Idempotent rollback | Finality for irreversible actions

Heads are sampled once per second and can skip intermediate tips. Subscription repair covers 32 blocks; reconnect and recover when the stream cannot repair a gap.

Head publication point Reorg reconstruction

Run it. Know the boundaries.

Native CLI · .NET 10 · Mainnet by default · Sepolia and Hoodi options

Start from the repository root

```
dotnet run --project src/Nethermind/Nethermind.LightClient -c
release -p:SaveDiskSpace=true -- --checkpoint-url
https://mainnet.checkpoint.sigp.io
```

Example uses Sigma Prime's checkpoint service as the trust anchor. Alternatively provide `--checkpoint <trusted-root>`.

HTTP `http://127.0.0.1:8545`

WS `ws://127.0.0.1:8545/ws`

32	256	16
state ancestors	block / receipt ancestors	finalized blocks / log range

Scope to communicate clearly

- `pending`, `safe` and `earliest` selectors are unavailable.
- Transaction lookup is bounded. `null` means unknown in scope, not global absence.
- Recent Electra-or-later checkpoints; this is light-client verification, not full-chain replay.

Remaining engineering work

- Peer diversity, CL capability selection and wider interoperability.
- QUIC/TLS conformance and lifecycle follow-up; native handler cursors/recovery.
- Browser/mobile ports and BAL-based cache advancement remain future work.

Local wallets work by default. Browser extension origins are allowed.

Public RPC needs an authenticated gateway.